Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Digital Assignment Lab - 2

Course Name :- Agile Development Process and Devops Lab

Course code :- ISWE406P

Name :- C P . Jyotheswar Reddy

Reg-No:- 23MIS0097

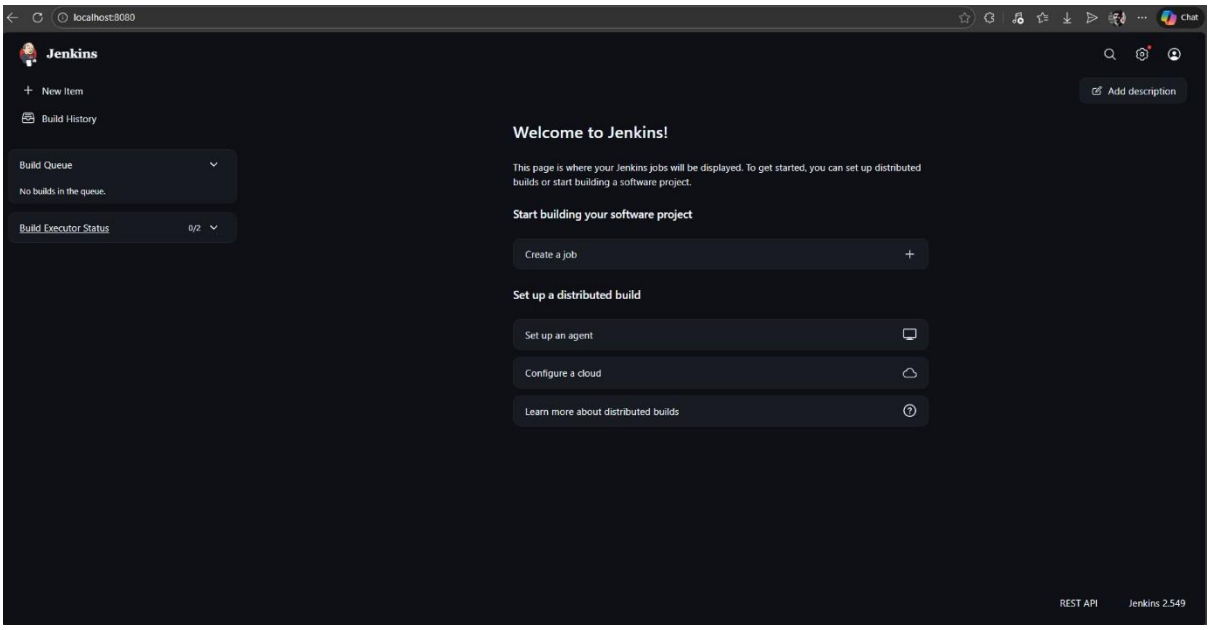
1. Learning Goal Understand what Jenkins is and run a job successfully. Stepwise Tasks

1.What is CI/CD? (Real-world example)

CI/CD stands for Continuous Integration and Continuous Delivery/Deployment.
Continuous Integration (CI):
Developers frequently push code to a shared repository (GitHub/GitLab).
Jenkins automatically:
1)Pulls the latest code
2)Builds the project

3)Runs unit tests
4)Reports success or failure
continuous Delivery/Deployment (CD):
After successful build and tests, Jenkins automatically deploys the application to:
1)Test server
2)Staging server
3)Production server

2. **Install Jenkins (Windows / Linux)**



3. **Jenkins UI overview**

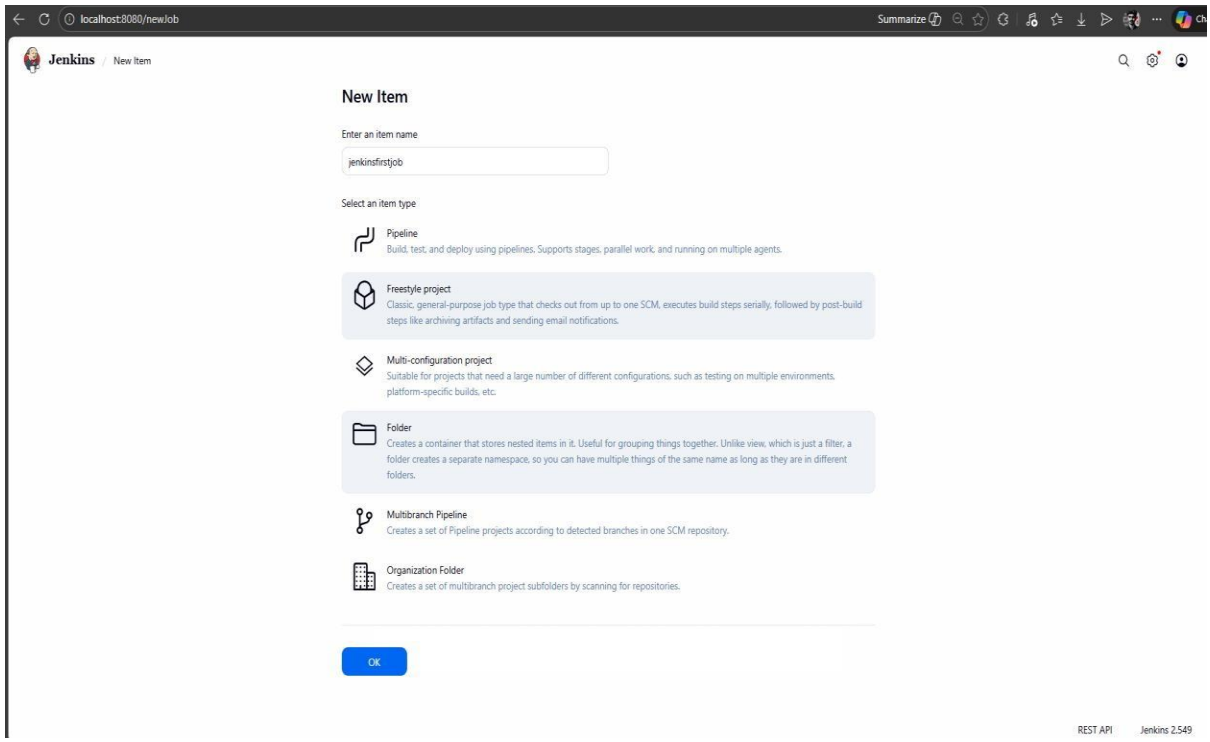
Main Components Visible in Jenkins UI

1. **Welcome Dashboard** o Displays the message *"Welcome to Jenkins!"* o Central area shows options to start creating jobs and configure agents.
2. **New Item** o Used to create:
   - Freestyle project
   - Pipeline job
   - Multibranch pipeline o This is the starting point for any automation task.
3. **Build History** o Shows the list of previously executed builds.
   - Helps to track:
     - Success
     - Failures

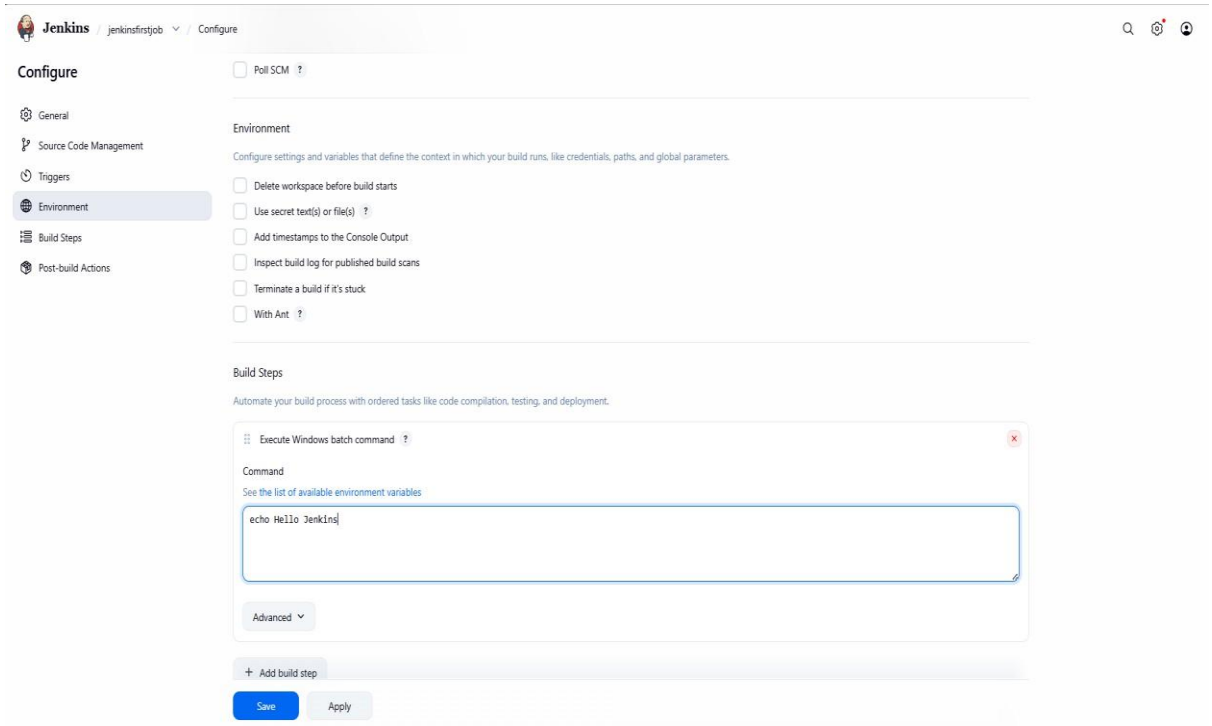
- Build numbers and timestamps.

4. **Build Queue** o Displays jobs that are waiting to be executed.
   - Useful when multiple builds are triggered at the same time.
5. **Build Executor Status** o Shows how many executors (agents) are available and busy.
   - Helps monitor Jenkins performance.
6. **Create a Job** o Quick option in the center panel to create a new project.
7. **Set up an Agent / Cloud** o Used to configure distributed builds o Jenkins can run jobs on remote machines (agents).
8. **Manage Jenkins (Settings Icon)** o Used for:
   - Plugin installation
   - User management
   - Security settings
   - Tool configuration.
9. **Search & User Profile**
   - Top-right corner provides:
     - Search option
     - Logged-in user details
     - System configuration.

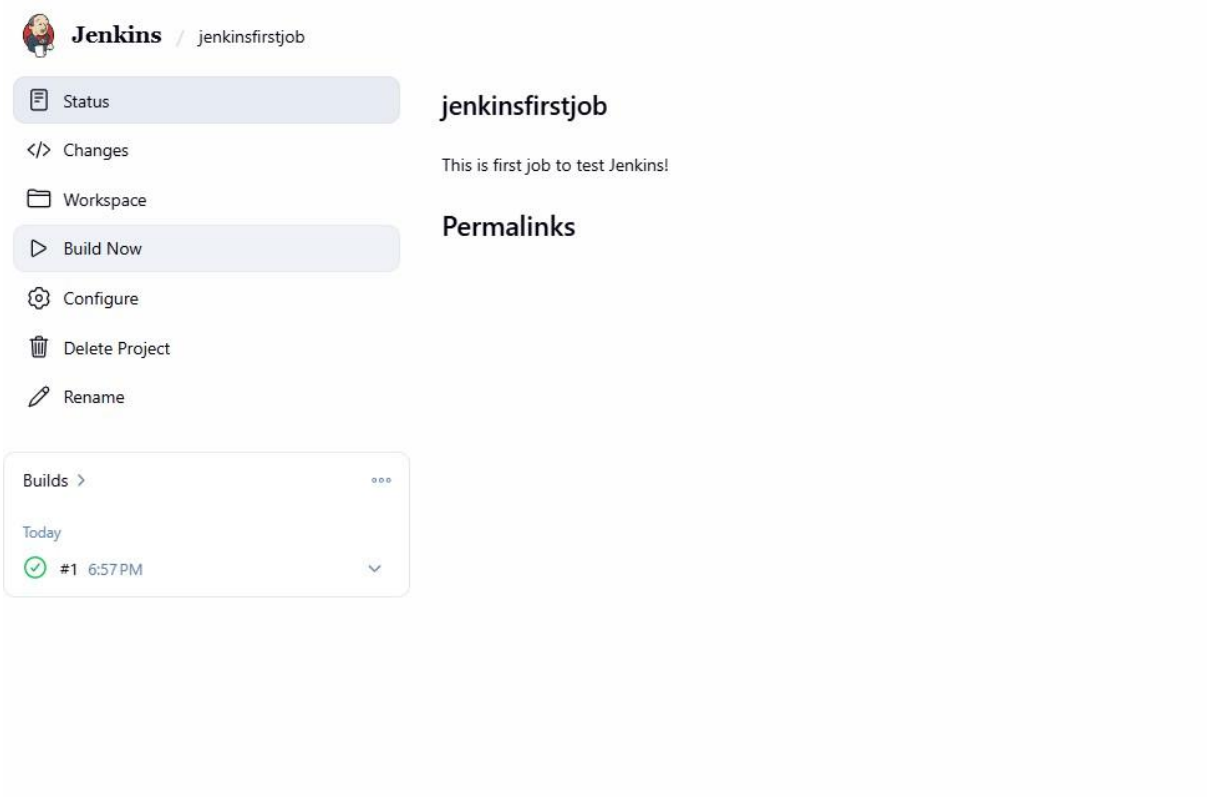
4. Create Freestyle Job



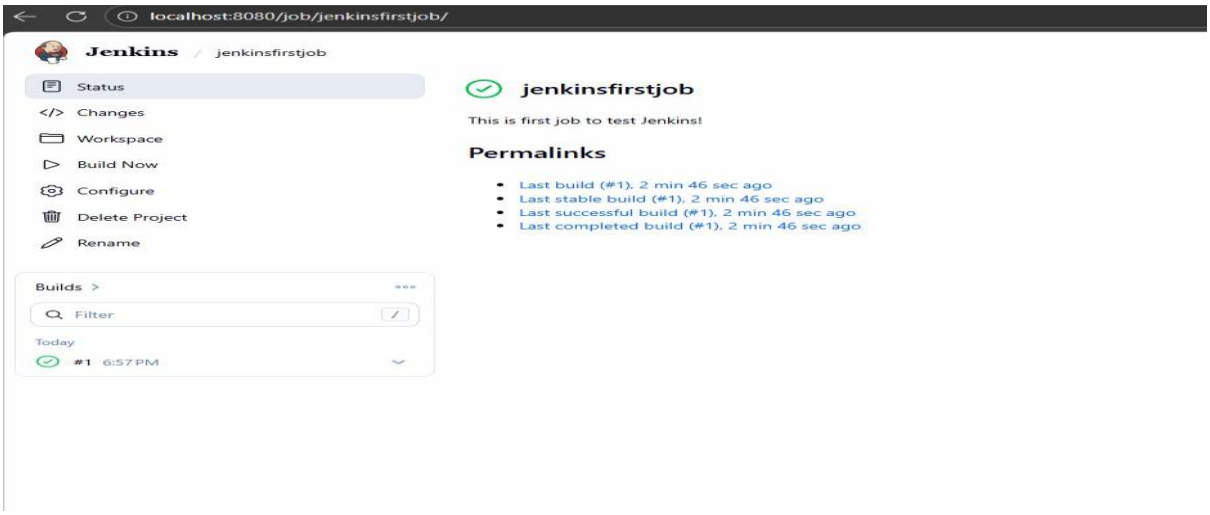
5. Add simple build step: Windows: echo Hello Jenkins



6. Run job manually



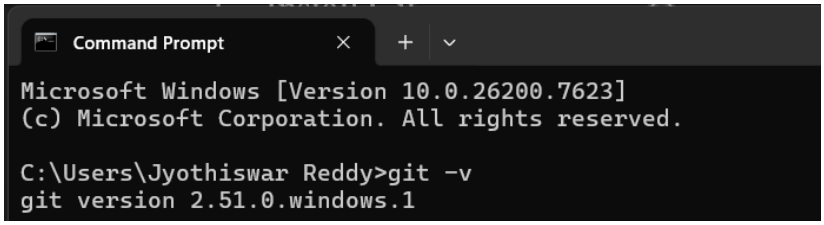
7. View: o Console Output o Build History



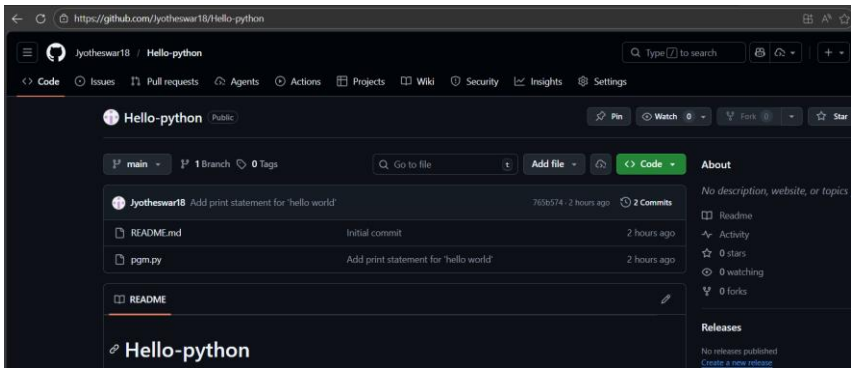
2. Learning Goal Work with source code and build automation

Stepwise Tasks

1. Install Git

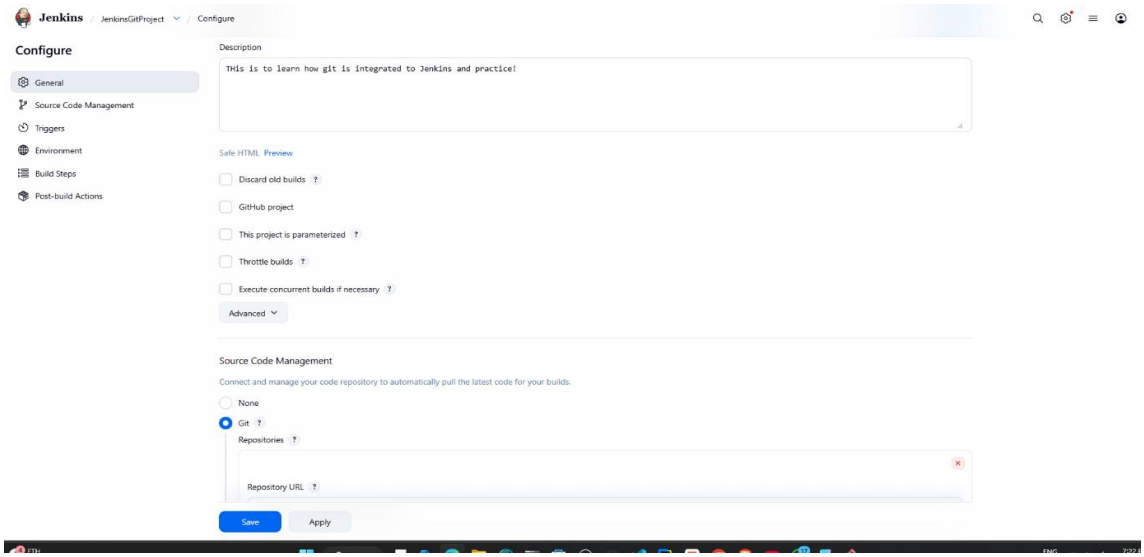


2. Create GitHub repository

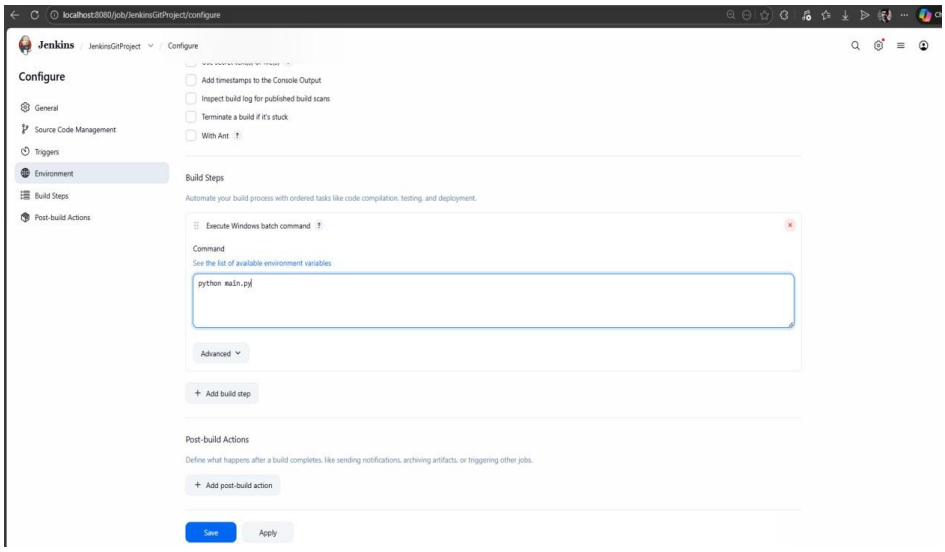


3. Configure Git in Jenkins

4. Create Freestyle Job: o Source Code Management → Git



5. Add build step: o Windows batch / Shell



6. Poll SCM / Webhook trigger

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- [ ] Trigger builds remotely (e.g., from scripts) ?
- [ ] Build after other projects are built ?
- [ ] Build periodically ?
- [ ] GitHub hook trigger for GITScm polling ?
- [x] Poll SCM ?

Schedule ?

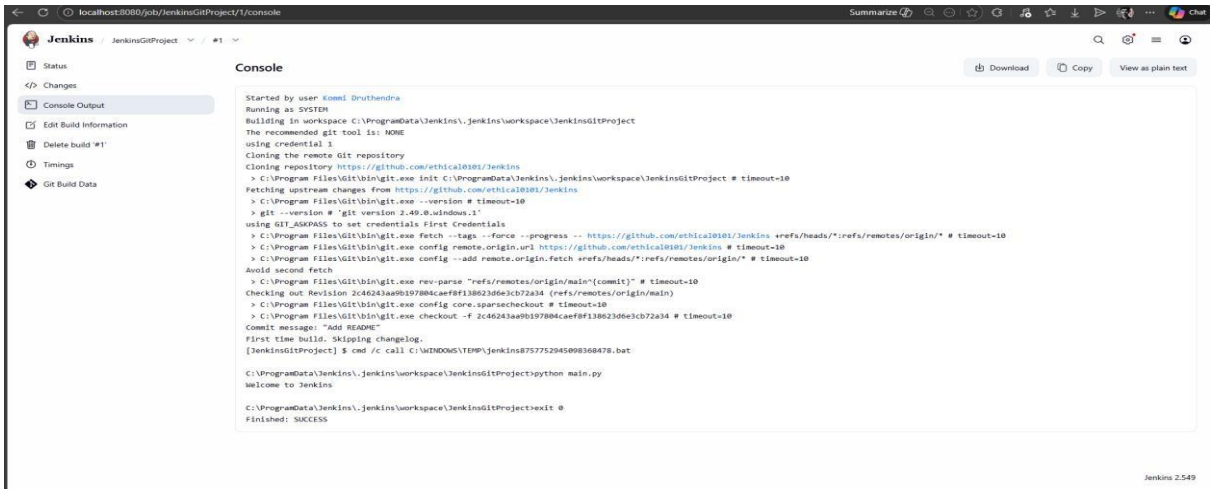
```
* * * * *
```

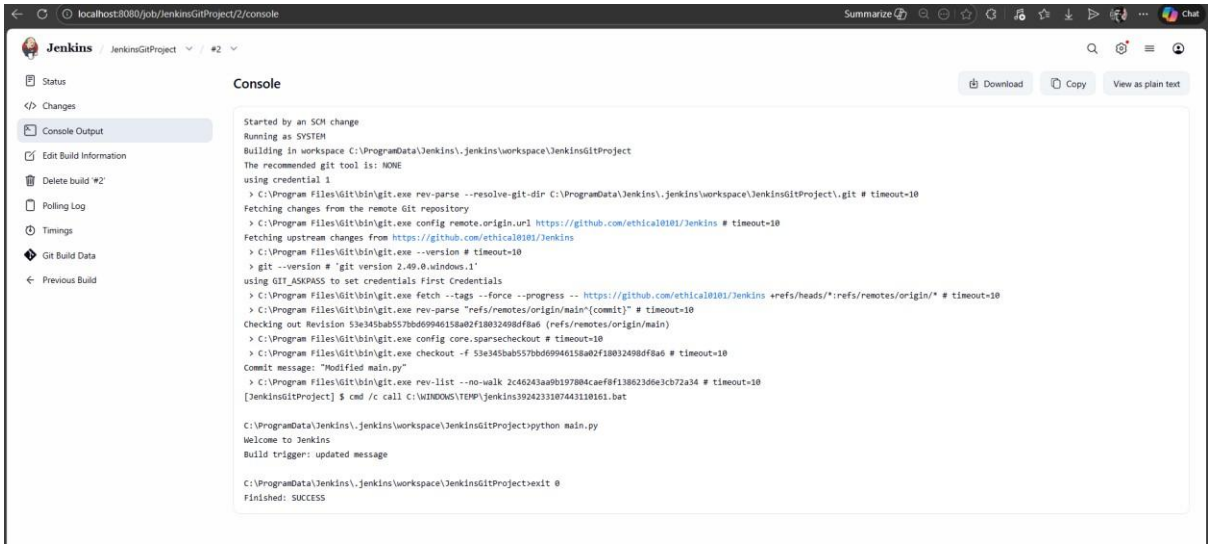
⚠ Do you really mean "every minute" when you say "* * * * *"? Perhaps you meant "H * * * *" to poll once per hour

Would last have run at Thursday, 5 February 2026, 7:25:00 pm India Standard Time; would next run at Thursday, 5 February 2026, 7:26:00 pm India Standard Time.

- [ ] Ignore post-commit hooks ?

7. Build automatically on code change





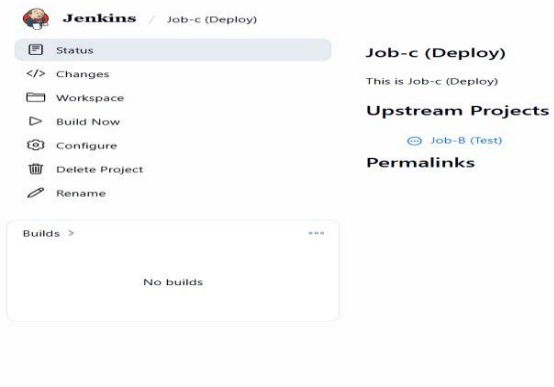
3). Learning Goal Understand job flow visually Stepwise Tasks

1. Create multiple Freestyle jobs:

o Job-A (Build)

o Job-B (Test)

o Job-C (Deploy)



2. Use: o Post-build action → Trigger other projects

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

Build other projects ?

Projects to build

Job-B (Test)

- (●) Trigger only if build is stable
- ( ) Trigger even if the build is unstable
- ( ) Trigger even if the build fails

+ Add post-build action

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

Build other projects ?

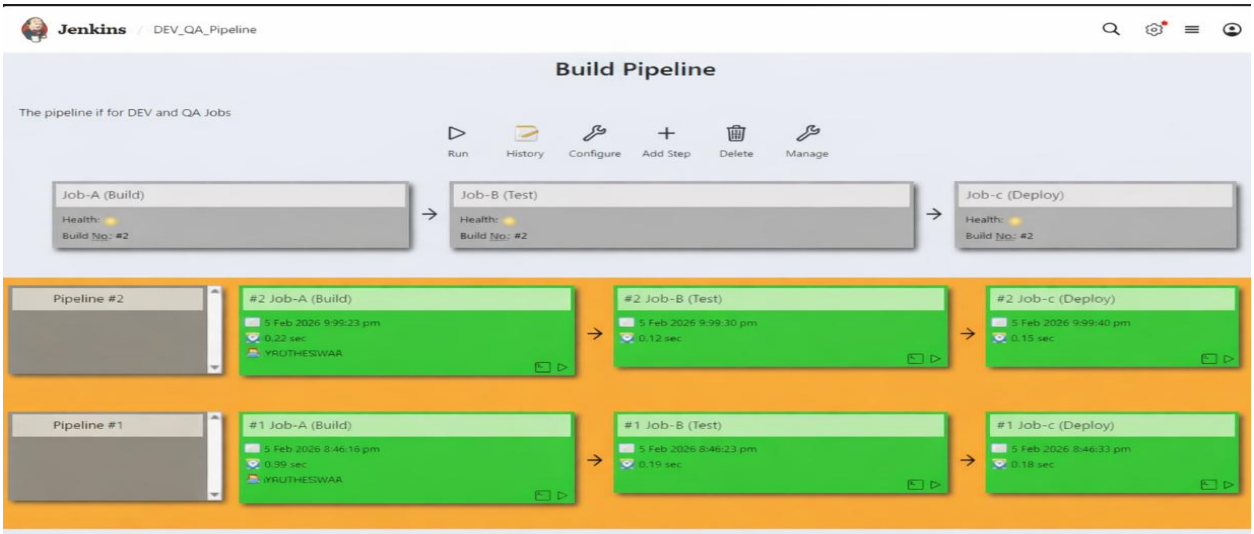
Projects to build

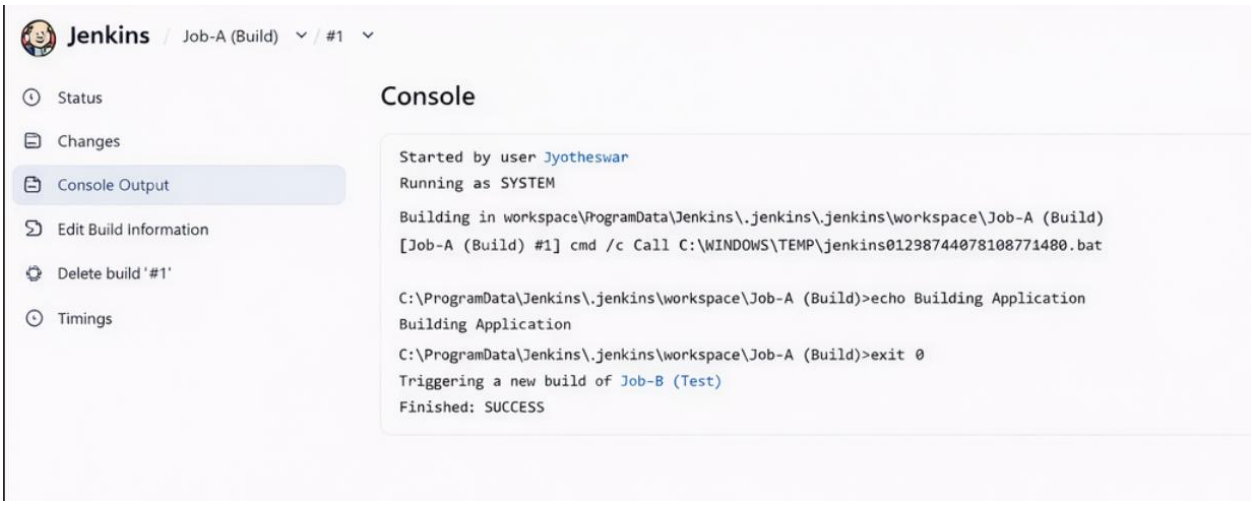
Job-c (Deploy)

- (●) Trigger only if build is stable
- ( ) Trigger even if the build is unstable
- ( ) Trigger even if the build fails

+ Add post-build action

3. View job chaining visually





4. Install Pipeline Stage View

4. Learning Goal Use Jenkins Pipeline & Jenkinsfile

Stepwise Tasks

1. What is Jenkins Pipeline?

   A **Jenkins Pipeline** is a suite of automated steps that defines the complete CI/CD process such as:

   - Build
   - Test
   - Deploy

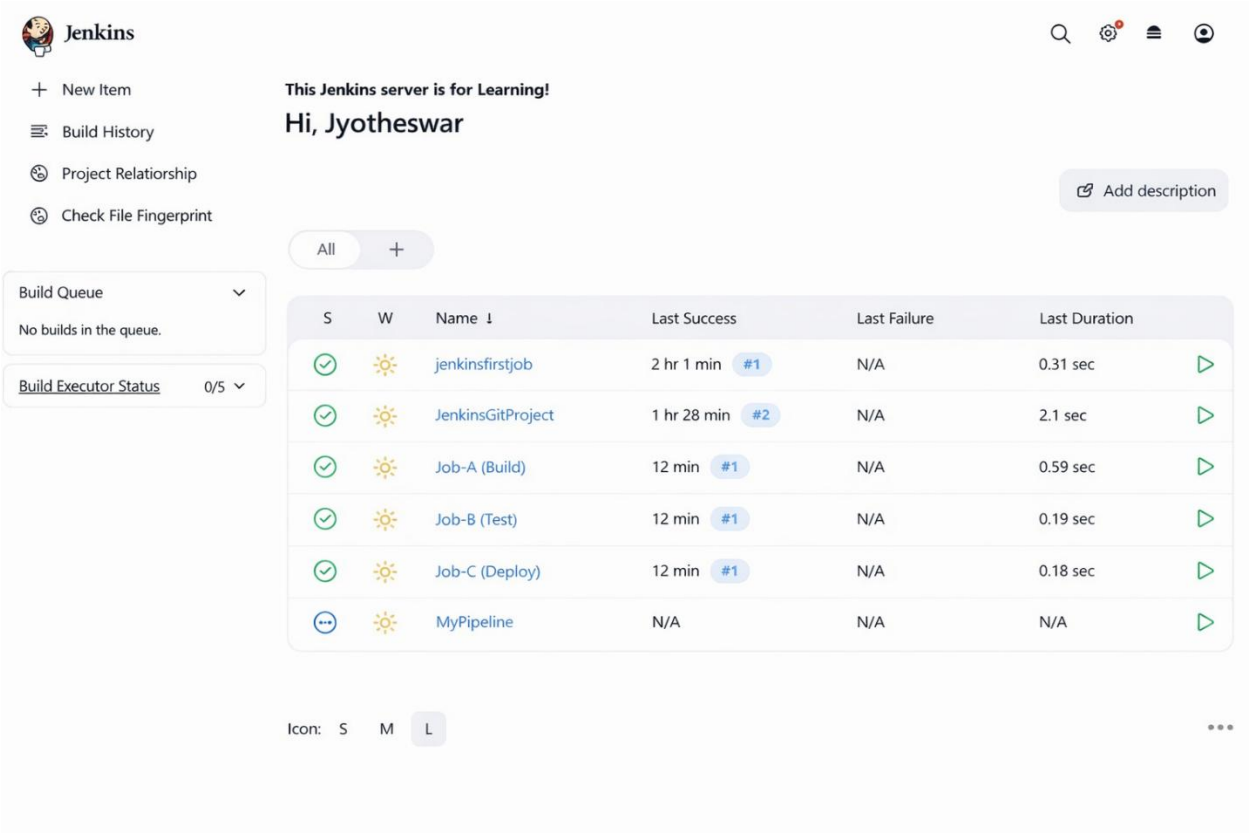
   It allows the entire workflow to be written as code and executed in Jenkins. Pipeline provides:

   - Automation of software delivery
   - Stage-wise execution
   - Better visualization
   - Reusability and version control

2. Scripted vs Declarative Pipeline

Feature	Scripted Pipeline	Declarative Pipeline
Syntax	Groovy based	Structured syntax
Complexity	More complex	Simple & readable
Recommended	Advanced users	Beginners & industry
Error handling	Manual	Built-in

3. Create Pipeline Job



4. Write pipeline in Jenkins UI

```
pipeline {   agent any

  stages {
```

stage('Build')

{ steps {

echo 'Build

Stage' }

}

stage('Test') { steps { echo 'Test Stage'

}

}

stage('Deploy') { steps { echo 'Deploy Stage'

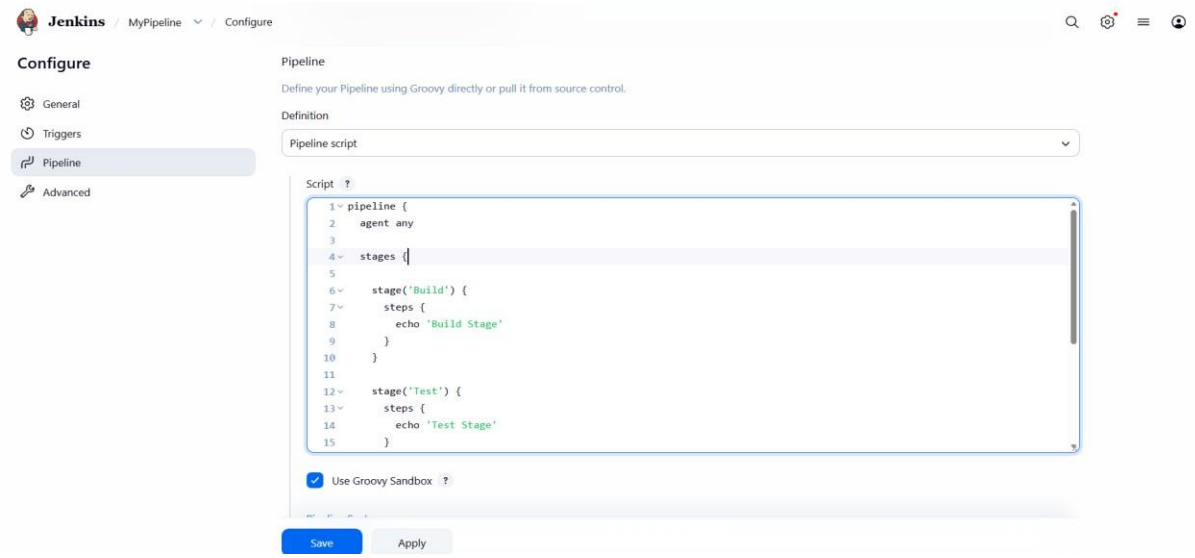
}

}

}

}

5. Add stages: o Build o Test o Deploy



6. Introduce: o agent o stages o steps Sample Task pipeline

pipeline

{

```
agent any
 stages {
     stage('Build') {
                steps {
                echo 'Build Stage'
     }
    }
stage('Test') {
       steps {
                  echo 'Test Stage'
     }
    }
 stage('Deploy') {
     steps {
                echo 'Deploy Stage'
     }
    }
  }
}
}
```

5. Learning Goal Pipeline as code from repository

Stepwise Tasks

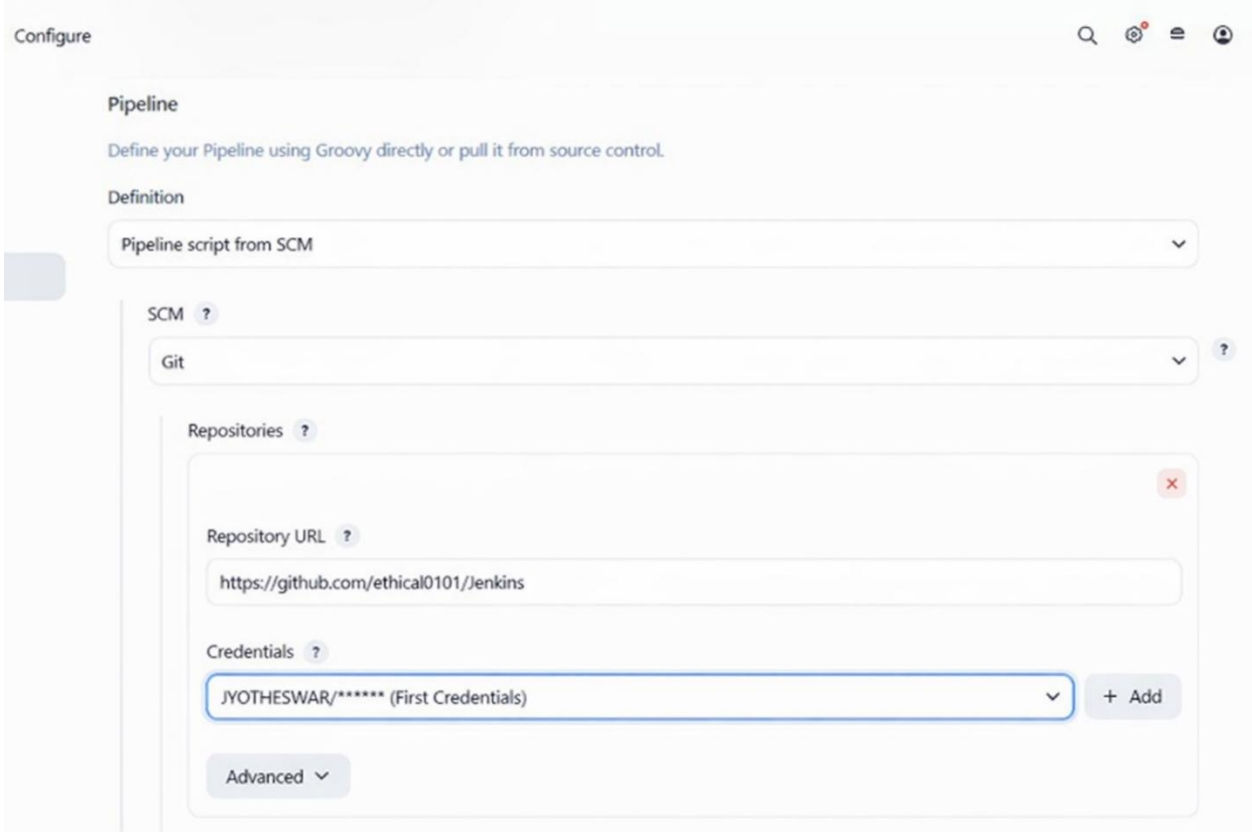
1. Create Jenkinsfile

```
pipeline {
  agent any
  stages {

    stage('Build') {
      steps {
        echo 'Building from GitHub'
      }
    }

    stage('Test') {
      steps {
        echo 'Testing from GitHub'
      }
    }

    stage('Deploy') {
      steps {
        echo 'Deploying from GitHub'
      }
    }

  }
}
```

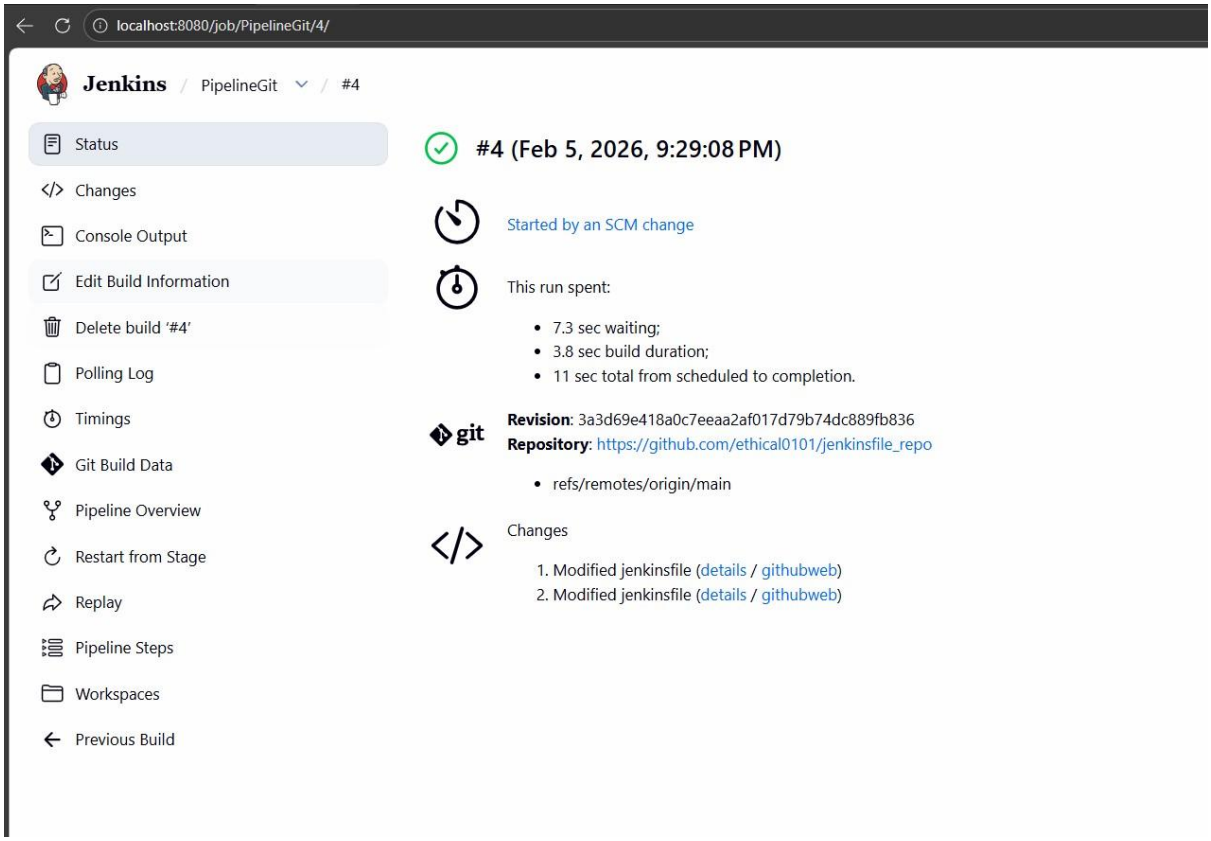
2. Commit Jenkinsfile to GitHub

```
PS E:\Jenkins-Practice-repo> git add .
PS E:\Jenkins-Practice-repo> git commit -m "Modified jenkinsfile"
[main 0968803] Modified jenkinsfile
 1 file changed, 1 insertion(+), 1 deletion(-)
PS E:\Jenkins-Practice-repo> git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 365 bytes | 365.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ethical0101/Jenkins.git
   8e42b98..0968803  main -> main
```

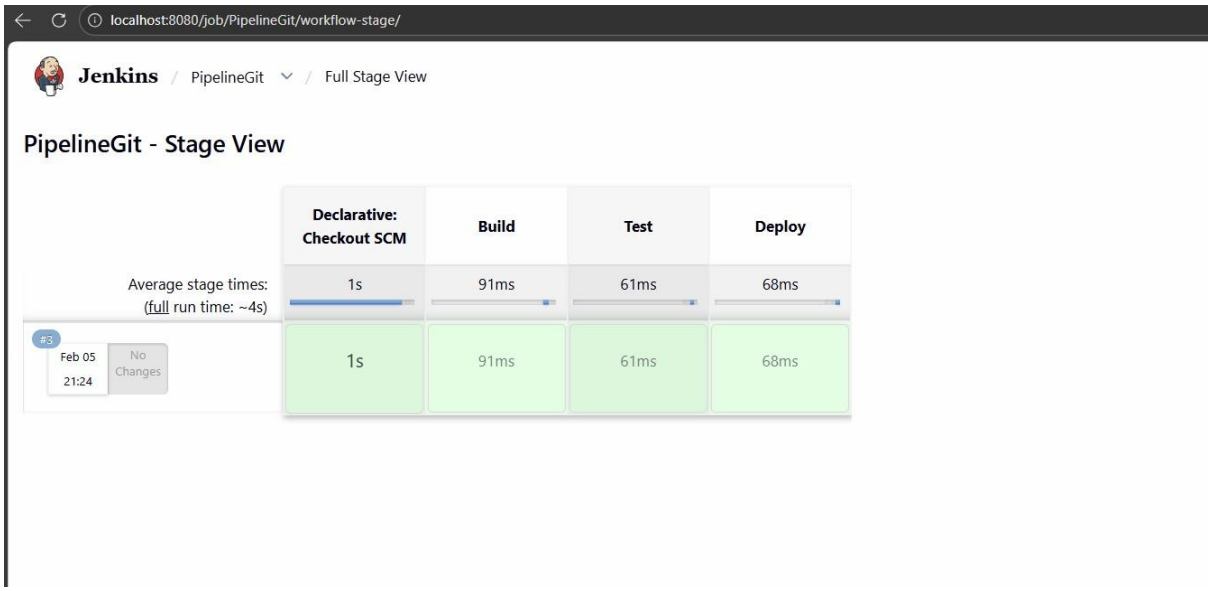
3. Configure Pipeline job: o SCM → Git



4. Trigger build automatically



5. Modify Jenkinsfile and rebuild

```
PS E:\jenkinsfile_Repo> git add .
PS E:\jenkinsfile_Repo> git commit -m "Modified jenkinsfile"
[main 4c3d4a5] Modified jenkinsfile
 1 file changed, 1 insertion(+), 1 deletion(-)
PS E:\jenkinsfile_Repo> git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 288 bytes | 288.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ethical0101/jenkinsfile_repo.git
   1fab71e..4c3d4a5  main -> main
```

6. View Stage View